

Guia de Apresentação — Busca e Ordenação (G09)

Responsável: Luiz Henrique de Moraes Chagas

Grupo: G09 · **Contexto:** Farmácia

Arquivo relacionado: sistema.c (busca, linhas ~272–311; ordenação, linhas ~313–334)

Antes de começar

Este guia cobre **só a sua parte**: busca sequencial, busca binária e Bubble Sort. Outros colegas cuidam de fila, pilha, lista e relatório.

Estrutura	Algoritmo aplicado
Lista → vetor (op. 3)	Busca sequencial por senha
Histórico → vetor (op. 7)	Bubble Sort por senha

A busca binária está **implementada** mas **não aparece no menu** — cite como conhecimento extra (exige vetor ordenado).

Trace manual detalhado: TRACE-MANUAL.pdf (mesma pasta).

Abertura sugerida

*"Fui responsável pelos **algoritmos de busca e ordenação**. Na opção 3, buscamos clientes com busca sequencial e contamos comparações. Na opção 7, o Bubble Sort ordena o histórico e contamos trocas — requisito único do G09."*

Ordem sugerida na apresentação

- Contexto — por que busca sequencial + Bubble Sort (README questão 3)
- listaParaVetor (Henrique) → buscaSequencial — opção 3
- Contagem de comparações impressa
- ordenar — Bubble Sort (explicar com trace manual)

5. Opção 7 com Gabriel — "X trocas realizadas"
 6. (Opcional) buscaBinaria — extra, vetor precisa estar ordenado
-

Funções — o que explicar

buscaSequencial

Quando roda: opção 3, após listaParaVetor (Henrique).

Passo a passo:

1. Inicializa comparacoes = 0
2. Percorre o vetor do índice 0 até n-1
3. A cada elemento, incrementa comparações
4. Se `vet[i].senha == senha`, imprime comparações e retorna i
5. Se não achou, imprime comparações e retorna -1

Frase de apoio: "Percorre elemento a elemento — adequado para poucos cadastros."

Exemplo: vetor [003, 002, 001], busca senha 002 → 2 comparações.

buscaBinaria

Quando roda: não está no menu (implementação extra).

Passo a passo:

1. low = 0, high = n - 1
2. Enquanto low <= high, calcula mid
3. Compara senha do meio; ajusta low ou high
4. Retorna índice ou -1

Frase de apoio: "Só funciona em vetor **ordenado** — por isso usamos sequencial na lista, que não é ordenada."

ordenar

Quando roda: dentro de gerarRelatorio (opção 7, Gabriel).

Passo a passo:

1. Dois for aninhados (Bubble Sort)
2. Compara `vet[j].senha` com `vet[j+1].senha`

3. Se maior, troca e incrementa contador trocas
4. Imprime "%d trocas realizadas\n"

Frase de apoio: "Compara apenas senha. Gestante não entra na comparação — só define quem chegou ao histórico antes."

Use o trace: abra TRACE-MANUAL.pdf e percorra as passagens ao vivo.

Demonstração no terminal

```
make && ./main
```

```
Opção 1 – cadastrar Maria, Joao, Ana
Opção 3 – buscar senha 002 (mostrar "X comparacoes feitas")
Opção 1 – cadastrar mais 2 (Carla gestante, Pedro)
Opção 2 – atender 3 vezes (montar histórico para trace)
Opção 5 – histórico antes de ordenar (Vinicius)
Opção 7 – relatório + "2 trocas realizadas" (com Gabriel)
```

Perguntas do README / diário

Questão 3 — Adequação dos algoritmos

São adequados porque o sistema lida com volumes pequenos: fila até 11 clientes e histórico de 3 atendimentos — compatível com o fluxo de uma farmácia.

Frase de apoio: "Poucos registros — não precisamos de algoritmos mais complexos."

Questão 5 — Estabilidade

O Bubble Sort é **estável**, mas **não importa** neste contexto: cada cliente tem senha única. Não há elementos iguais para preservar ordem relativa.

FAQ / armadilhas

Pergunta	Resposta
O que é o trace manual?	Bubble Sort na op. 7 — ver TRACE-MANUAL.pdf

Pergunta	Resposta
Por que sequencial e não binária na op. 3?	Lista não é ordenada; vetor copiado mantém ordem de inserção
O que conta como troca?	Cada vez que dois clientes adjacentes são permutados
Ordena a pilha original?	Não — gerarRelatorio ordena cópia em vetor auxiliar

Checklist antes da apresentação

- ☐ Compilar: make
- ☐ Testar opções 3 e 7
- ☐ Ter TRACE-MANUAL.pdf impresso ou aberto
- ☐ Saber explicar questões 3 e 5 do README
- ☐ Combinar com Henrique (op. 3) e Gabriel (op. 7)